

ExpenseSense: Privacy-Preserving On-Device Personal Finance Tool Calling with Sub-2B LLMs

Anonymous ACL submission

Abstract

For personal finance, cloud-based inference poses privacy risks, highlighting the need for local on-device execution. Yet, the tool-calling performance of sub-2B small language models (SLMs) remains under-explored. We introduce **ExpenseSense**, a privacy-first personal finance assistant, and benchmark on-device tool calling using a 115-question test suite. We compare single-agent (joint routing and parameter extraction) against dual-agent (router-specialist) pipelines across six local models (0.8B–2B). Dual-agent decomposition improves correctness (e.g., Qwen3.5-2B Correct Ratio improves from 60.2% to 77.6%), as well as decreases average latency by up to 48% (e.g., from 12.0 s to 6.2 s for Qwen3.5-2B). It also shifts errors toward safer failure modes. Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy with 3.0 s latency and 1.2 GB RAM, demonstrating the viability of fully private, on-device financial assistants. All code, data, and prompts are available on GitHub.

1 Introduction

Large language models (LLMs) have rapidly advanced tool-augmented AI (Schick et al., 2023; Patil et al., 2023; Qin et al., 2024), enabling agents to invoke structured APIs from natural language. However, research primarily targets large cloud-hosted models, which is problematic for personal finance where local inference is crucial for privacy. Expense records reveal sensitive behavioral patterns, yet users expect natural language interfaces (e.g., asking “how much did I spend on food last month?”) that map to structured queries over local data (Li et al., 2024). This creates an underexplored problem: can *sub-2B quantized models* running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial language?

Existing benchmarks (Patil et al., 2025; Qin et al., 2024; Zhu et al., 2026; Cao et al., 2026) fo-

cus on models above 7B parameters, which are prohibitively slow for local deployment. Conversely, on-device models like Octopus v2 (Chen and Li, 2024) and Hammer (Lin et al., 2024) demonstrate the potential of smaller models but have not been evaluated on domain-specific financial schemas or under consumer-hardware latency constraints.

We address three research questions (RQ):

- **RQ1:** Can sub-2B quantized SLMs achieve reliable personal finance analytics tool calling on consumer hardware, enabling fully private local inference?
- **RQ2:** Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy gains over a joint single-agent architecture?
- **RQ3:** What are the primary failure modes across architectures, and how do they differ qualitatively?

This paper introduces **ExpenseSense**, with the following contributions:

1. A **115-question dataset** for financial tool calling across five analytical tools. We introduce a 7-feature complexity scoring function to categorize queries into three difficulty tiers (L1–L3), simulating real-world interactions.
2. A comparative evaluation of **single-agent vs. dual-agent** architectures across six quantized SLMs (0.8B–2.0B parameters), showing that while optimized single-agent prompts can achieve 92.0% Task Accuracy, dual-agent decomposition reduces latency (by up to 48% for Qwen3.5-2B), improves exact-match correctness (+17.4 pp Correct Ratio), and stabilizes sub-1B models without relying on cloud resources.
3. A **fine-grained error taxonomy** of 7 categories, showing that dual-agent architectures

consolidate errors into predictable failure modes, which is highly desirable for safety-critical financial systems.

4. Empirical **on-device deployment guidelines** for privacy: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy with 3.0 s latency and ≈ 1.2 GB RAM, proving that private, high-quality financial analytics can run locally on consumer hardware.

2 Privacy Motivation

The deployment of LLMs in personal finance highlights the conflict between AI utility and user privacy. Cloud-based services require users to transmit detailed spending information to external servers. Even if the underlying database is kept local, user queries can leak sensitive details: a question like “how much did I spend on medical expenses last month?” discloses both related spending and temporal patterns. [Staab et al. \(2024\)](#) demonstrated that LLMs can infer personal attributes like income, location, and health conditions from seemingly innocuous text, and [Yao et al. \(2024\)](#) catalog privacy risks including data leakage, memorization, and inference attacks.

ExpenseSense addresses this threat by adopting an *on-device, privacy-first* architecture. By leveraging sub-2B quantized models that run efficiently on consumer hardware (tested on an Apple M1 with 8 GB unified memory), all user data and queries remain strictly local. No financial records, query text, or model outputs leave the device. This local-first approach aligns with [Nissenbaum’s](#) framework of *contextual integrity*: financial data generated for personal expense tracking should remain within that context rather than flowing to third-party providers with different informational norms.

The key challenge is that on-device inference with sub-2B models has historically been unreliable for structured tool calling ([Kavathekar et al., 2025](#)). Our work demonstrates that architectural decomposition—separating intent classification from parameter extraction—bridges this reliability gap, making private on-device inference practical.

3 Related Work

Tool-Augmented Language Models. Toolformer ([Schick et al., 2023](#)) demonstrated self-supervised tool-use learning; Gorilla ([Patil et al., 2023](#)) and ToolLLM ([Qin et al., 2024](#)) scaled

to thousands of real-world APIs. HuggingGPT ([Shen et al., 2023](#)) and ToolAlpaca ([Tang et al., 2023](#)) explored multi-step orchestration. APIGen ([Liu et al., 2024b](#)) introduced a verified pipeline for generating function-calling training data, showing that 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. These approaches, however, rely on cloud-hosted models ($\geq 7B$), leaving the privacy-constrained on-device setting largely unaddressed.

Function-Calling Evaluation. The Berkeley Function Calling Leaderboard ([Patil et al., 2025](#)) established a benchmark using AST substring matching and execution-based validation. [Kavathekar et al. \(2025\)](#) proposed an evaluation suite (FSP, ACS, AVC, Task Accuracy, and Correct Ratio) to evaluate small model function calling without code execution. We adopt these metrics directly. CallNavi ([Song et al., 2025](#)) proposed a two-stage router+specialist approach—similar to our dual-agent design—but evaluates models $\geq 7B$ on general-purpose APIs. In contrast, we focus on sub-2B models for domain-specific, single-turn financial tasks under strict privacy constraints.

On-Device Function Calling. MobileLLM ([Liu et al., 2024a](#)), Phi-3 ([Abdin et al., 2024](#)), and related surveys ([Xu et al., 2024](#)) have explored sub-4B parameter models for mobile deployment. Octopus v2 ([Chen and Li, 2024](#)) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer ([Lin et al., 2024](#)) introduced function masking to reduce overfitting to function names, addressing an issue similar to the category normalization errors we observe. Apple Intelligence ([Gunter et al., 2024](#)) validated on-device tool calling at scale with a $\sim 3B$ model, demonstrating the viability of the paradigm. [Kavathekar et al. \(2025\)](#) found that structured output compliance is the primary bottleneck for SLMs, which motivates our dual-agent architecture.

Financial NLP and Privacy. FinQA ([Chen et al., 2021](#)) and FinGPT ([Yang et al., 2023](#)) target financial numerical reasoning and generation, but focus on text generation rather than tool calling under privacy constraints. FinMCP-Bench ([Zhu et al., 2026](#)) and FinTrace ([Cao et al., 2026](#)) evaluate LLM agents for financial tool use, but target cloud-scale models. Personal finance assistants ([Li](#)

Tool	ID	Key Params
plot_time_series	1	category, year/month, ranges, months, ignore_rent
plot_distribution	2	category, year/month/day, ranges, months, ignore_rent
plot_comparison_bars	3	category, y1/m1, y2/m2, sm/em/ey, ignore_rent
calculate_total	4	category, remarks, year/month/day, months, ignore_rent
get_top_expenses	5	n, category, year/month/day, months, min_amount, ignore_rent

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

2. **Specialist:** The predicted tool ID selects a tool-specific prompt containing 12–15 examples for that *one* tool, along with detailed parameter definitions and date-handling rules.

The router sees $\approx 1,000$ tokens; the specialist sees $\approx 1,500$ – $2,000$ tokens with deeper, more directly relevant supervision. This simplifies the task for each model call and provides more relevant in-context examples, though it requires two sequential inference passes. Combined prompt tokens: $\approx 2,700$ per query.

4.3 Post-Hoc Parameter Validation

A post-processing validation layer corrects category names deterministically without additional model calls:

1. **Exact match** against the known vocabulary.

2. **Morphological normalisation:** groceries $\rightarrow$ grocery, cafés $\rightarrow$ cafe.

3. **Edit-distance matching** via difflib (cutoff ≥ 0.80).

4. **Token overlap scoring** (Jaccard; cutoff ≥ 0.50).

All results report both raw (pre-validation) and validated (post-validation) metrics.

5 Benchmark Design

5.1 Test Suite

The benchmark comprises 115 test cases across five task groups (Table 2). Queries are written in naturalistic, informal language: abbreviations (“compare electricity ’24 vs ’25”), colloquialisms (“can ya tell me”), misspellings (“distribuution”), and domain-specific terms (“combinis”, “nomikais”, “shinkansen”). This mimics the Japan-based personal finance context of the target application and tests robustness to spelling variations, colloquialisms, and cultural terms.

5.2 Complexity Scoring

Each test case is assigned a difficulty score:

$$C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

where n_p = number of expected parameters; $d_{\text{date}} \in \{0-4\}$ scores date specification complexity; $d_{\text{cat}} \in \{0-2\}$ is the category normalisation

Task Group	Cases	Avg Params	Avg C
Time Series	23	2.9	7.0
Distribution	21	2.5	6.4
Comparison	28	4.4	8.8
Calculate Total	22	2.4	6.3
Top Expenses	21	3.2	6.7
Total	115	3.2	7.1

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

Model	Family	Params	RAM
EXAONE-4.0-1.2B	EXAONE-4.0 (2025)	1.2B	2.0 GB
Gemma-3-1B-it	Gemma-3 (2025)	1B	1.6 GB
LFM2.5-1.2B	LFM2.5 (2026)	1.2B	1.4 GB
MiniCPM5-1B	MiniCPM (2024)	1B	1.5 GB
Qwen3.5-0.8B	Qwen3.5 (2026)	0.8B	1.3 GB
Qwen3.5-2B	Qwen3.5 (2026)	2B	2.1 GB

Table 3: Models evaluated. All use Q8_0 GGUF quantization via llama.cpp. RAM is average peak RSS.

distance; $d_{\text{rel}} \in \{0, 1\}$ flags relative time expressions; $d_{\text{multi}} \in \{0-2\}$ counts compound parameter groups; $d_{\text{abbr}} \in \{0, 1\}$ flags abbreviated dates; and $d_{\text{hol}} \in \{0, 2\}$ flags calendar knowledge requirements. Scores are binned into three tiers: **L1** ($C \leq 4$, 24 cases), **L2** ($5 \leq C \leq 7$, 45 cases), **L3** ($C \geq 8$, 46 cases).

5.3 Evaluation Metrics

We adopt the metric suite of Kavathekar et al. (2025):

- **FSP** (Function Selection Precision): Binary indicator of correct tool prediction.
- **ACS** (Argument Coverage Score): F_1 on argument *name* overlap.
- **AVC** (Argument Value Correctness): Fraction of correctly named arguments whose values also match.
- **Task Accuracy**: F_1 on the combined set $\{\text{tool}\} \cup \{k=v\}$, jointly capturing tool selection and parameter correctness.
- **Correct Ratio (CR)**: Strict binary indicator—1.0 only when Task Acc = 1.0, i.e. the entire function call is perfectly correct.

All metrics are averaged across 5 repetitions per test case (temperature = 0.1) with 95% bootstrap confidence intervals (Efron, 1979) ($n = 2000$ resamples).

5.4 Error Taxonomy

Each prediction is classified into one of seven mutually exclusive error types: CORRECT, TOOL_WRONG, PARSE_FAILURE, PARAM_MISSING, PARAM_VALUE_WRONG, PARAM_EXTRA, FORMAT_VIOLATION. This taxonomy helps diagnose specific failure modes rather than just aggregate accuracy, which is critical for evaluating safety profile of on-device financial assistants.

6 Experimental Setup

6.1 Models

All six models (Table 3) use Q8_0 GGUF quantization and run via llama.cpp (Gerganov, 2023) with full GPU offload on Apple Metal.

6.2 Hardware and Inference Protocol

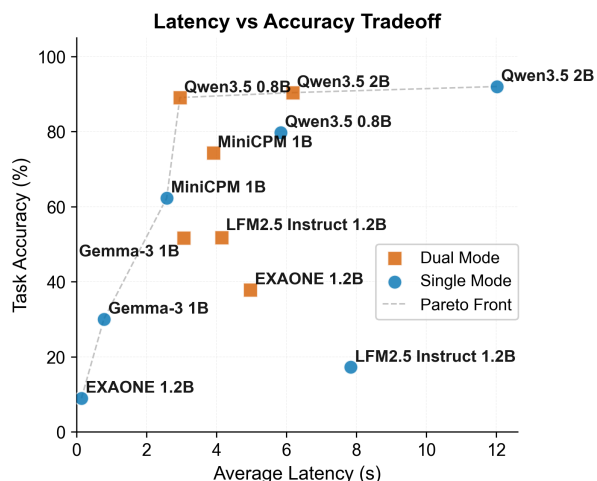
Experiments run on an Apple M1 (8-core CPU, 7-core GPU, 8 GB unified memory)—a consumer laptop representative of privacy-constrained deployment environments. Parameters: temperature = 0.1, min_p = 0.05, n_ctx = 4096, stop tokens `["\n\n", "\'"]`. Each model is warmed up through all six prompt templates (one router + five specialists) before evaluation begins. Each of the 115 test cases is run 5 times per model per architecture, yielding $115 \times 5 \times 6 \times 2 = 6,900$ total observations.

7 Results

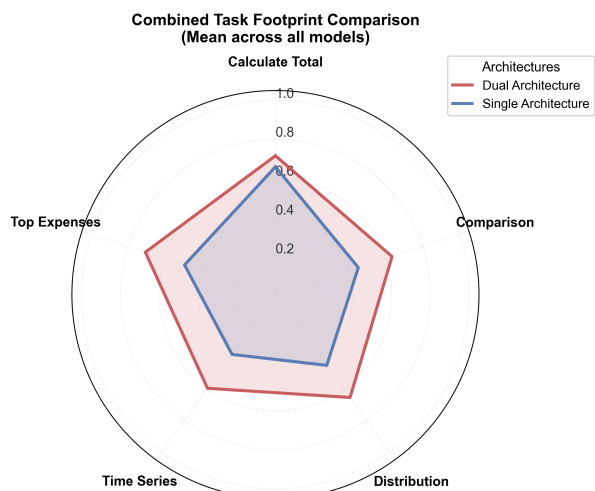
7.1 Overall Performance

Table 4 presents the full results. Figure 1 shows the accuracy–latency Pareto frontier and the error taxonomy breakdown.

Key findings (RQ1, RQ2). (1) Sub-2B SLMs can reliably call financial tools on consumer hardware (RQ1), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This shows that cloud reliance can be avoided while keeping data private. (2) While optimized single-agent prompts allow Qwen3.5-2B to reach 92.0% Task Accuracy, dual-agent decomposition is still preferred for two reasons (RQ2): (a) Strict correctness—Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, representing a significant reduction in silent parameter er-



(a) Accuracy–latency Pareto frontier. The dual-mode Qwen3.5-0.8B (orange square, ≈ 3 s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at $4\times$ the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 1: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all task categories.

326 rors that could produce incorrect financial analyt-
 327 ics. (b) Enabling smaller models—Qwen3.5-0.8B
 328 improves from 79.8% to 89.1% Task Accuracy
 329 (+9.3 pp) while halving latency (5.8 s to 3.0 s)
 330 in dual mode, making sub-1B models viable on
 331 low-memory devices.

332 7.2 Per-Tool-Group Analysis

333 Table 5 shows per-group accuracy for the top two
 334 dual-mode models. Figure 2 shows the full model
 335 $\times$ category heatmap. `get_top_expenses`
 336 reaches 98.9% accuracy with Qwen3.5-2B due
 337 to its simple parameter structure. Conversely,
 338 `plot_time_series` is the most difficult task
 339 (74.4% for Qwen3.5-2B) due to complex date spec-
 340 ifications and relative time expressions (e.g., “past
 341 X months”) requiring temporal reasoning.

342 7.3 Complexity Tier Analysis

343 Figure 3 presents the per-model accuracy break-
 344 down across complexity tiers in dual mode. Task
 345 Accuracy decreases from 96% (L1) to 82% (L3) for
 346 Qwen3.5-2B, and from 95% to 84% for Qwen3.5-
 347 0.8B. L3 queries involve constructs like cross-year
 348 range comparisons (“*compare groceries nov 2024*”
 349 *to apr 2025 vs nov 2025 to apr 2026*”), which re-
 350 quire extracting up to 10 parameters with complex
 351 temporal constraints.

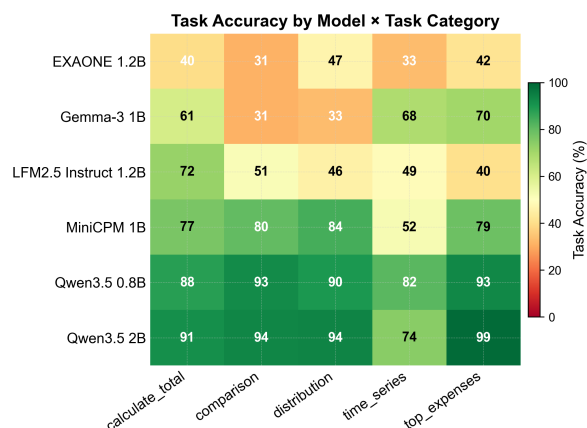


Figure 2: Task Accuracy (%) by Model $\times$ Tool Category in dual mode. Qwen3.5 models achieve $>74\%$ across all categories, while weaker models show pronounced gaps in comparison and time series tasks.

352 7.4 Error Taxonomy Analysis

353 Figure 4 visualises the full error distribution. The
 354 error distribution shifts significantly between con-
 355 figurations (RQ3).

For Qwen3.5-2B: single-agent produces 22.6%
 PARAM_EXTRA and 7.3% PARAM_MISSING;
 dual-agent reduces PARAM_EXTRA to 1.9% while
 PARAM_MISSING remains at 8.0%. Eliminating
 extra parameters is critical for safety: for exam-
 ple, a spurious `months=1` filter added to “how
 much did I spend on food?” would restrict the
 query to the last month, returning an incorrect total.

Mode	Model	FSP	ACS	AVC	Task Acc	CR	Latency (s)	RAM (GB)
Single	EXAONE-4.0-1.2B	19.3	—	0.3	8.9	0.3	0.1	2.0
	Gemma-3-1B-it	27.0	—	47.5	30.0	0.9	0.8	1.6
	LFM2.5-1.2B	28.7	—	11.7	17.2	3.1	7.8	1.4
	MiniCPM5-1B	71.1	—	86.7	62.3	13.0	2.6	1.5
	Qwen3.5-0.8B	95.0	—	91.2	79.8	40.2	5.8	1.3
	Qwen3.5-2B	96.5	—	97.6	92.0	60.2	12.0	2.1
Dual	EXAONE-4.0-1.2B	90.6	3.7	3.3	37.8	3.0	5.0	1.9
	Gemma-3-1B-it	55.1	59.9	71.6	51.7	16.5	3.1	1.5
	LFM2.5-1.2B	69.6	51.6	57.2	51.8	16.3	4.1	1.4
	MiniCPM5-1B	84.9	77.0	81.6	74.3	43.5	3.9	1.5
	Qwen3.5-0.8B	94.3	90.2	96.8	89.1	61.7	3.0	1.2
	Qwen3.5-2B	92.0	91.6	97.4	90.4	77.6	6.2	2.1

Table 4: Main results (validated metrics, %). **Bold** = best per metric. FSP = Function Selection Precision; ACS = Argument Coverage Score; AVC = Argument Value Correctness; Task Acc = F_1 -based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (—) where the single-agent architecture’s joint output makes isolated argument coverage less meaningful. Latency and RAM are per-query averages.

Group	Qwen3.5-0.8B		Qwen3.5-2B	
	T.Acc	CR	T.Acc	CR
Time Series	81.6	54.8	74.4	60.9
Distribution	89.6	56.2	93.8	79.0
Comparison	92.5	71.4	93.7	73.6
Calc. Total	87.9	59.1	91.4	84.5
Top Expenses	93.3	64.8	98.9	92.4

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

Model (Dual)	Δ Task Acc	Δ CR
EXAONE-4.0	+0.00	+0.00
Gemma-3-1B-it	+0.12	+0.00
LFM2.5-1.2B	+0.50	+0.00
MiniCPM5-1B	+1.17	+0.87
Qwen3.5-0.8B	+0.41	+1.57
Qwen3.5-2B	+0.29	+0.87

Table 6: Validation layer impact (Δ in percentage points, dual mode). Gains are modest but consistent for capable models.

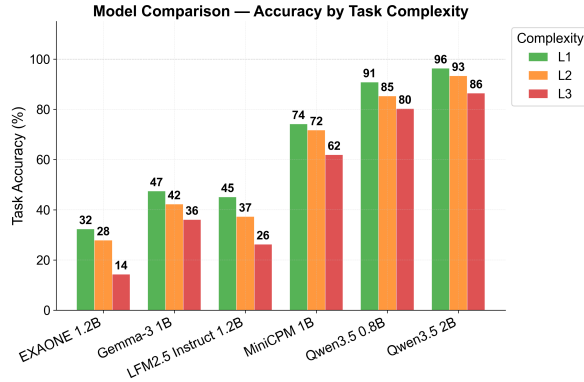


Figure 3: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

364 Dual-agent’s residual errors are primarily missing
365 parameters (PARAM_MISSING), which is a rela-
366 tively safer failure mode that simply omits a filter
367 rather than returning an incorrect subset.

368 For Qwen3.5-0.8B: single-agent produces 32.5%
369 PARAM_MISSING and 11.8% PARAM_EXTRA;
370 dual-agent reduces these to 12.9% and 11.1% re-
371 spectively, shifting the model from a high-variance
372 to a more predictable failure profile.

373 7.5 Token Efficiency

374 Figure 5 shows that despite using $\approx 2,700$ prompt
375 tokens per query (vs. $\approx 5,500$ for single-agent), the
376 dual-agent architecture achieves equivalent or supe-
377 rior accuracy. The single-agent pipeline processes
378 larger contexts but exhibits lower performance per
379 token since the model must simultaneously attend
380 to all five tool schemas and disambiguation instruc-
381 tions. The dual-agent architecture decomposes the
382 context, focusing each pass on a single subtask to
383 achieve higher accuracy-per-token.

384 7.6 Validation Layer Impact

385 The validation layer (Table 6) contributes mod-
386 est gains, primarily correcting category cas-
387 ing/normalisation. This reveals that argument ex-
388 traction rather than string normalization remains
389 the primary bottleneck, since validation cannot re-
390 cover missing parameters (PARAM_MISSING).

391 8 Discussion

392 **Privacy through Task Decomposition.** The
393 dual-agent architecture both improves accuracy and

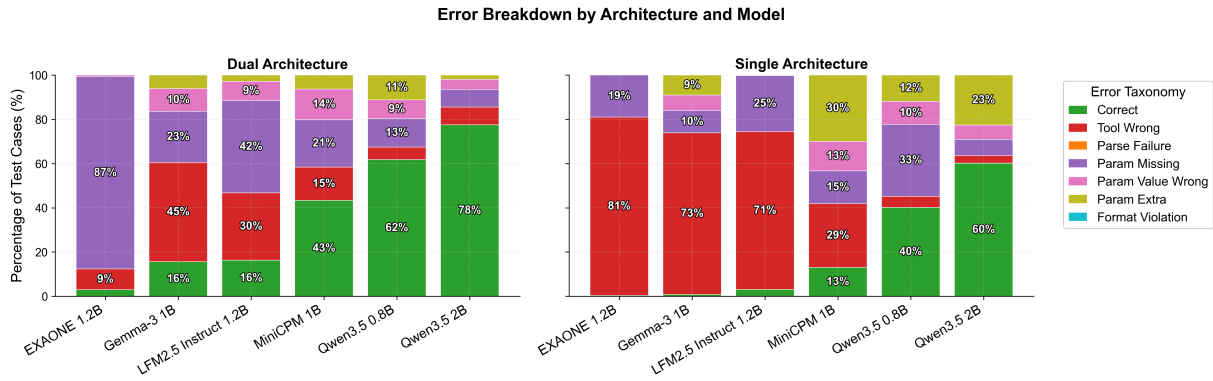


Figure 4: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B (right panel) produces 22.6% PARAM_EXTRA errors—spurious parameters that could lead to incorrect financial calculations. Dual-agent (left) eliminates most of these, consolidating residual errors into the more predictable PARAM_MISSING category.

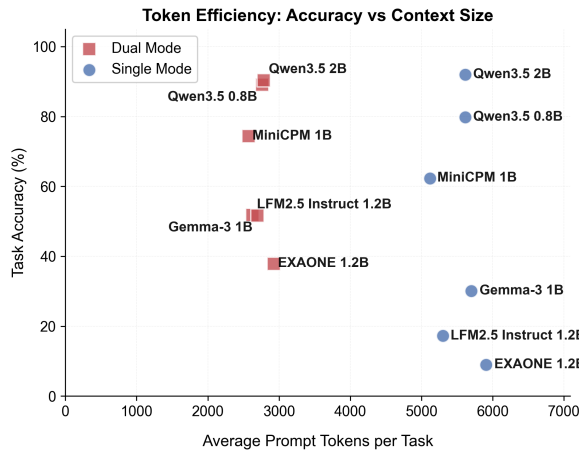


Figure 5: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue), demonstrating superior context utilisation.

410 provides a +9.3 pp Task Accuracy boost *and* halves
411 latency, because shorter, more focused prompts
412 generate faster on small models. (3) The error tax-
413 onomy shift from PARAM_EXTRA (dangerous in
414 finance) to PARAM_MISSING (safer) represents a
415 qualitative safety improvement.

416 **Analyzing the EXAONE Failure.** EXAONE-
417 4.0-1.2B achieves 90.6% FSP in dual mode—
418 excellent routing—but averages near-empty special-
419 ist outputs (ACS = 3.7%). This likely points
420 to formatting or parsing issues with the quantized
421 model’s template rather than a capacity limit. Fine-
422 tuning techniques, such as function masking or
423 functional tokens, could help resolve these format-
424 ting failures.

425 **Addressing Highly Complex Queries.** Even the
426 best model achieves only 59.6% strict Correct Ra-
427 tio on L3 queries in dual mode, which involve
428 cross-year range comparisons requiring up to 10
429 interdependent parameters. Fine-tuning on domain-
430 specific examples using verified synthetic data (Liu
431 et al., 2024b) is a potential avenue to improve per-
432 formance on complex queries.

394 makes local inference viable. By separating rout-
395 ing (single-digit output) from extraction (focused
396 JSON), the dual-agent setup reduces the genera-
397 tion complexity per call, addressing the primary
398 bottleneck of small models. Decomposing the task
399 helps sub-2B models perform reliably on domain-
400 specific schemas.

401 **Why dual-agent still matters despite strong**
402 **single-agent prompts.** Although prompt opti-
403 mization increases single-agent performance, dual-
404 agent configurations remain preferable for several
405 reasons: (1) The 17.4 pp gap in Correct Ratio
406 (60.2%→77.6%) means that single-agent produces
407 wrong parameters in 40% of queries vs. 22% for
408 dual-agent. (2) For the privacy-optimal Qwen3.5-
409 0.8B (which fits in <1.5 GB RAM), dual-agent

433 **Deployment Recommendations.** Based on
434 our evaluation, we recommend: (1) *Memory-*
435 *constrained* (≤ 2 GB available): Qwen3.5-0.8B
436 dual, 89.1% Task Acc, 3.0 s, 1.2 GB RAM. (2)
437 *Accuracy-first*: Qwen3.5-2B dual, 90.4% Task
438 Acc, 77.6% CR, 6.2 s, 2.1 GB RAM. (3) *Budget*
439 *hardware*: MiniCPM5-1B dual, 74.3% Task Acc,
440 43.5% CR, 3.9 s, 1.5 GB RAM. All configurations
441 keep user data entirely on-device.

Case Study. Figure 6 shows a usage session of ExpenseSense. A user queried the system in informal language—“can ya plot spending on snacks for past 14 months?”—which the dual-agent Qwen3.5-0.8B pipeline correctly routed to `plot_time_series` and extracted the parameters `{category: snacks, months: 14}`. The resulting visualization revealed a clear declining trend in weekly snack expenditure (from peaks of ¥3,000–4,000 to under ¥1,000), showing how local analytics can guide budgeting. This demonstrates that local models can process natural queries and provide actionable insights without transmitting data.

9 Conclusion

We presented ExpenseSense, a human-centered evaluation of privacy-preserving, on-device financial tool calling using sub-2B quantized models. We find that decomposing the task into intent routing and parameter extraction remains essential for sub-2B models: it improves strict correctness (from 60.2% to 77.6% Correct Ratio) and enables sub-1B models to reach 89.1% Task Accuracy with 3.0 s latency. Analysis of error types shows that dual-agent architectures also result in safer failure modes, shifting errors from incorrect parameter additions to missing parameters, which is crucial for financial applications.

Several directions warrant future investigation. First, *fine-tuning* the best-performing models on domain-specific function-calling data could address the residual L3 failures while preserving on-device deployment. Second, extending the benchmark to *multi-turn* interactions would capture realistic conversational patterns. Third, *multilingual* query support would broaden applicability across diverse user populations. Finally, evaluating lower-bit quantisation formats (Q4_K_M) could further reduce the memory footprint, making privacy-preserving financial AI accessible on even more constrained devices such as smartphones.

Limitations

- The benchmark targets a single expense tracking application; generalisability to other financial domains (trading, compliance, auditing) requires further investigation.
- All models use Q8_0 quantisation; lower-bit formats (Q4_K_M) may yield different accuracy–efficiency tradeoffs.

- Queries are in English and reflect a Japan-based personal finance context (categories such as “combinis”, “nomikais”, “shinkansen”). Multilingual and cross-cultural diversity is not covered.
- Ground-truth labels are deterministic; some queries may admit legitimate alternative tool selections.
- Test cases were authored by a single developer; crowdsourcing or user study collection would improve query diversity.
- We do not evaluate the *downstream correctness* of the financial analytics produced—only whether the tool call is correctly formed. Production deployment would require additional safeguards against calculation errors.
- Our privacy analysis is architectural (no data leaves the device) rather than cryptographic; we do not address model inversion or side-channel attacks on the local device itself.

Ethics Statement

ExpenseSense is designed for on-device inference specifically to preserve user financial privacy. No real user financial data was used in the benchmark; all test cases reference synthetic expense categories. The system is intended to complement, not replace, professional financial advice. We recognise that even on-device models may produce incorrect financial summaries, and advocate for clear user-facing disclaimers in production deployments.

Data and Code Availability

Code, prompts, benchmark data, and full results will be released on GitHub upon acceptance. Released artefacts include: all 115 test cases with ground-truth labels, the complete router and specialist prompt templates, inference configuration details, raw and validated CSV result files, and the benchmark runner, plotting scripts, and the exact model GGUFs used in this study to ensure complete reproducibility.

References

- Marah Abdin, Jyoti Aneja, Hany Awadalla, and 1 others. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.
- Yupeng Cao, Haohang Li, Weijin Liu, Wenbo Cao, Anke Xu, Lingfei Qian, and 1 others. 2026. FinTrace: Holistic trajectory-level evaluation of LLM

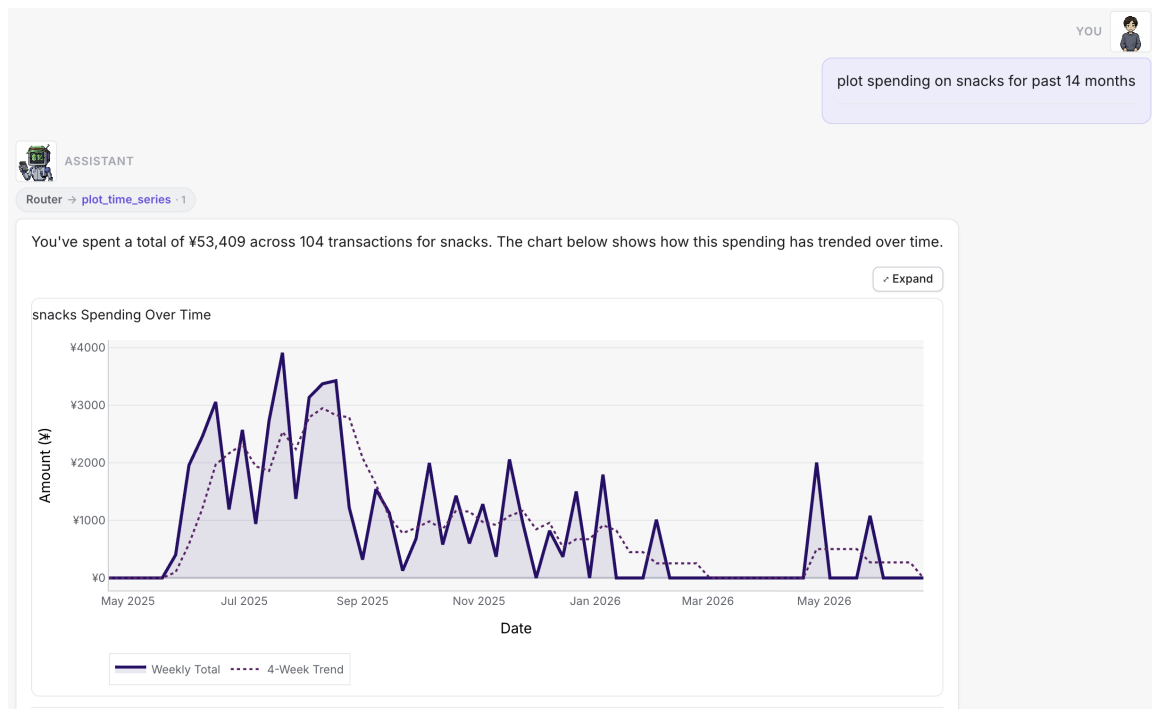


Figure 6: Real usage session: a user queries snack spending trends in informal language. The dual-agent Qwen3.5-0.8B pipeline correctly invokes `plot_time_series`, producing an actionable spending trend—entirely on-device with no data transmitted externally.

- 539 tool calling for long-horizon financial tasks. *arXiv* 566
540 preprint *arXiv:2604.10015*. 567
- 541 Wei Chen and Zhiyuan Li. 2024. Octopus v2: On- 569
542 device language model for super agent. *arXiv* 570
543 preprint *arXiv:2404.01744*. 571
- 544 Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena 572
545 Shah, Iana Borova, Dylan Langdon, Reema Moussa 573
546 Matt Beane, Ting-Hao Huang, Bryan Routledge, and 574
547 William Yang Wang. 2021. *FinQA: A dataset of* 575
548 *numerical reasoning over financial data*. In *Proceed-* 576
549 *ings of the 2021 Conference on Empirical Methods* 577
550 *in Natural Language Processing*, pages 3697–3711. 578
- 551 B. Efron. 1979. Bootstrap methods: Another look at the 579
552 jackknife. *The Annals of Statistics*, 7(1):1–26.
- 553 Gemma Team. 2025. Gemma 3 technical report. *arXiv* 580
554 preprint *arXiv:2503.19786*. 581
- 555 Georgi Gerganov. 2023. *llama.cpp*. 582
- 556 Tom Gunter, Zirui Wang, Chong Wang, and 1 others. 583
557 2024. Apple intelligence foundation language mod- 584
558 els. *arXiv preprint arXiv:2407.21075*. 585
- 559 Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu 587
560 Cui, Xiang Long, and 1 others. 2024. MiniCPM 588
561 Unveiling the potential of small language models 589
562 with scalable training strategies. *arXiv preprint* 590
563 *arXiv:2404.06395*. 591
- 564 Ishan Kavatkar, Raghav Donakanti, Ponnurangam Ku- 592
565 maraguru, and Karthik Vaidhyanathan. 2025. *Small* 593
- models, big tasks: An exploratory empirical study on small language models for function calling. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 1117–1126.
- LG AI Research. 2025. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. *arXiv preprint arXiv:2507.11407*.
- Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, and 1 others. 2024. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.
- Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, and 1 others. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587*.
- Liquid AI. 2026. LFM2.5 technical report. *Liquid AI Blog*.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yuniang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024a. *MobileLLM: Optimizing sub-billion parameter language models for on-device use cases*. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32431–32454.

595 Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, 549 Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao,
596 Tian Lan, and 1 others. 2024b. APIGen: Auto-550 Qi Cai, and Ziyuan Ling. 2024. On-device language
597 mated pipeline for generating verifiable and diverse551 models: A comprehensive review. *arXiv preprint*
598 function-calling datasets. *Advances in Neural Infor-552 arXiv:2409.00088*.
599 mation Processing Systems, 37.

600 Helen Nissenbaum. 2004. Privacy as contextual in553 Hongyang Yang, Xiao-Yang Liu, and Christina Dan
601 tegrity. *Washington Law Review*, 79(1):119–158. 554 Wang. 2023. *FinGPT: Open-source financial large*
602 Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Char555 *language models*. In *International Symposium on*
603 lie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and 556 *Large Language Models for Financial Services (Fin-*
604 Joseph E. Gonzalez. 2025. *The Berkeley function557 LLM 2023) at IJCAI 2023*.
605 calling leaderboard (BFCL): From tool use to agentic558 Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo
606 evaluation of large language models. In *Proceedings559 Sun, and Yue Zhang. 2024. A survey on large lan-*
607 of the 42nd International Conference on Machine560 guage model (LLM) security and privacy: The good,
608 Learning, pages 48371–48392. 561 the bad, and the ugly. *High-Confidence Computing*,
609 Shishir G. Patil, Tianjun Zhang, Xin Wang, and 562 4(2):100211.
610 Joseph E. Gonzalez. 2023. *Gorilla: Large language563 Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi*
611 model connected with massive APIs. *arXiv preprint564 Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng*
612 arXiv:2305.15334. 565 Chen, Yong Liu, and Chi Zhang. 2026. FinMCP-
613 Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan 566 Bench: Benchmarking LLM agents for real-world
614 Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, 567 financial tool use under the model context protocol.
615 Bill Qian, and 1 others. 2024. *ToolLLM: Facilitat-568 arXiv preprint arXiv:2603.24943*.
616 ing large language models to master 16000+ real-
617 world APIs. In *Proceedings of the Twelfth Inter-
618 national Conference on Learning Representations
619 (ICLR 2024)*.

620 Qwen Team. 2026. *Qwen3.5: Towards native multi-
621 modal agents*. Qwen Blog.

622 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta
623 Raileanu, Maria Lomeli, Eric Hambro, Luke Zettle-
624 moyer, Nicola Cancedda, and Thomas Scialom. 2023.
625 Toolformer: Language models can teach themselves
626 to use tools. In *Advances in Neural Information Pro-
627 cessing Systems*, volume 36.

628 Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li,
629 Weiming Lu, and Yueting Zhuang. 2023. Hugging-
630 GPT: Solving AI tasks with ChatGPT and its friends
631 in Hugging Face. In *Advances in Neural Information
632 Processing Systems*, volume 36.

633 Yewei Song, Xunzhu Tang, Cedric Lothritz, Saad Ezzini,
634 Jacques Klein, Tegawendé F. Bissyandé, Andrey
635 Boytsov, Ulrick Ble, and Anne Goujon. 2025. *Call-
636 Navi: A challenge and empirical study on LLM func-
637 tion calling and routing*. In *Proceedings of the 29th
638 International Conference on Evaluation and Assess-
639 ment in Software Engineering*.

640 Robin Staab, Mark Vero, Mislav Balunović, and Mar-
641 tin Vechev. 2024. Beyond memorization: Violating
642 privacy via inference with large language models. In
643 *Proceedings of the 12th International Conference on
644 Learning Representations*.

645 Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han,
646 Qiao Liang, and Le Sun. 2023. ToolAlpaca: Gener-
647 alized tool learning for language models with 3000
648 simulated cases. *arXiv preprint arXiv:2306.05301*.

669 A Complexity Scoring Features

Feature	Range	Description
n_p	0+	Number of expected parameters
d_{date}	0–4	Date complexity: none / year / month / range / specific day
d_{cat}	0–2	Category distance: exact / alias / semantic
d_{rel}	0–1	Relative time expression (“past X months”)
d_{multi}	0–2	Compound parameter groups
d_{abbr}	0–1	Abbreviated year formats (“’24”)
d_{hol}	0–2	Holiday/calendar knowledge required

Table 7: Seven features of the complexity scoring function (Eq. 1).

670 B Sample Test Cases

ID	Query	Tool	Expected Params	Tier
TS01	Can ya show spending on food for past 6 months?	plot_time_series	category=Food, months=6	L2
DI06	show breakdown of expenses for june 21 2024	plot_distribution	year=2024, month=6, day=21	L2
CP24	compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026	plot_comparison_bars	category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4	L3
CT07	sum total spent on metro for july 2024?	calculate_total	category=tokyo metro, year=2024, month=7	L2
TP06	top 5 expenses past month? disregard rent	get_top_expenses	n=5, months=1, ignore_rent=true	L2

Table 8: Sample test cases. Note informal language, abbreviations, and semantic mappings (“metro”→“tokyo metro”, “groceries”→“grocery”).

671 C Qualitative Error Examples

672 Table 9 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

Error Type	Query	Expected	Model Output
PARAM_MISSING	“Show spending trend for the last 3 months”	{months: 3}	{ } — months omitted in the relative-time case
PARAM_VALUE_WRONG	“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”	{em1: 4, em2: 4}	{em1: 11, em2: 11} — confuses start month for end month in cross-year range
PARAM_EXTRA	“How much did I spend on food?”	{category: Food}	{category: Food, months: 1} — spurious time filter restricts scope

Table 9: Representative failure cases for Qwen3.5-2B (dual mode). Note that PARAM_EXTRA is particularly dangerous in financial contexts: the spurious months=1 filter silently restricts “total food spending” to “last month’s food spending”.